

Ficheros y directorios

2. Gestión de ficheros y directorios

La gestión de ficheros y directorios se realiza a través de **Path** y **Files**.

- **Path**: Representa una **ruta** en el sistema de ficheros (ej. `/home/usuario/foto.png` o `C:\usuarios\docs\informe.txt`). Un objeto `Path` es una dirección y no significa que el fichero o directorio exista.

Métodos principales de Path

Método	Descripción
<code>Path.of(String)</code>	Crea un objeto <code>Path</code> a partir de un <code>String</code> de ruta (Java 11+). Por debajo llama a <code>Paths.get()</code> que es el método original de la clase <code>Paths</code> (Java 7+).
<code>toString()</code>	Devuelve la ruta como un <code>String</code> (se llama por defecto desde <code>println</code>).
<code>toAbsolutePath()</code>	Devuelve la ruta absoluta del <code>Path</code> .
<code>fileName()</code>	Devuelve el nombre del fichero o directorio final de la ruta.

Ejemplo 1

El siguiente código demuestra cómo crear y mostrar distintos tipos de rutas (no intenta acceder a los ficheros o directorios, por tanto pueden existir o no):

```
1 import java.nio.file.Path
2
3 fun main() {
4     rutas()
5 }
6
7 fun rutas() {
8     // Path relativo al directorio del proyecto (carpeta de muestras)
9     val rutaRelativa: Path = Path.of("muestras", "orquidea.jpg")
10
11     // Path absoluto en Windows
```



```
12     val rutaAbsolutaWin: Path = Path.of("C:", "Herbario", "Especies",
13     "Helechos")
14
15     // Path absoluto en Linux/macOS
16     val rutaAbsolutaNix: Path =
17     Path.of("/home/botanico/jardin/flora_mediterranea")
18
19     println("Ruta relativa: " + rutaRelativa) // Muestra la ruta relativa
20     println("Ruta absoluta: " + rutaRelativa.toAbsolutePath()) // Ruta
21     completa
        println("Ruta absoluta Windows: " + rutaAbsolutaWin)
        println("Ruta absoluta Linux: " + rutaAbsolutaNix)
    }
```



Prueba y analiza el ejemplo

Prueba el código de ejemplo y verifica que la salida por consola es:

```
1 Ruta relativa: muestras\orquidea.jpg
2 Ruta absoluta: D:\kot\ficheros\muestras\orquidea.jpg
3 Ruta absoluta Windows: C:\Herbario\Especies\Helechos
4 Ruta absoluta Linux: /home/botanico/jardin/flora_mediterranea
```



Autoevaluación

Pregunta 1: ¿Cuál será el comportamiento del siguiente código al ejecutarse si en el disco duro no existe ninguna carpeta llamada `datos` ni ningún archivo `videojuegos.csv` ?

```
1 import java.nio.file.Path
2
3 fun main() {
4     val ruta = Path.of("datos", "videojuegos.csv")
5     println("Ruta: $ruta")
6 }
```

- A) El programa lanzará una excepción en tiempo de ejecución (`NoSuchFileException`) al intentar crear un `Path` de un archivo que no existe en disco.
- B) El código se ejecutará correctamente y mostrará: `Ruta: datos\videojuegos.csv` (Windows) o `Ruta: datos/videojuegos.csv` (Linux/macOS).
- C) Se producirá un error de compilación porque `Path.of()` requiere que se pase un único String completo con las barras de separación.
- D) El programa se ejecutará pero imprimirá un valor vacío (`Ruta:`), ya que el sistema no puede resolver la dirección.

” Solución

- ☒ A) Un objeto `Path` representa únicamente una dirección lógica en el sistema de ficheros. Crear un `Path` no interactúa con el disco duro ni verifica si el fichero existe realmente.
- ☒ B) `Path.of` une los fragmentos de texto con el separador del sistema operativo. El código se ejecuta con éxito e imprime la ruta, exista o no el fichero en disco.
- ☒ C) `Path.of` está sobrecargado y acepta múltiples argumentos (`vararg`) para construir la ruta de forma independiente del sistema operativo. La sintaxis es totalmente válida.
- ☒ D) El objeto `Path` almacena la ruta como texto estructurado. Al usarlo en `println`, se llama a su `.toString()`, que devuelve la ruta formateada.

Pregunta 2: ¿Qué método de `Path` devuelve la ruta absoluta a partir de una ruta relativa?

- A) `toString()`
- B) `fileName`
- C) `toAbsolutePath()`
- D) `resolve(other)`



Solución

✗ A) `toString()` convierte la ruta a cadena de texto con el separador del sistema operativo, pero no la convierte en absoluta.

✗ B) `fileName` devuelve únicamente el último componente de la ruta (el nombre del fichero o directorio final), no la ruta completa.

✓ C) `toAbsolutePath()` combina la ruta relativa con el directorio de trabajo actual del proyecto y devuelve la ruta absoluta completa.

✗ D) `resolve(other)` une dos rutas (añade una subcarpeta o fichero a un `Path` base), pero no convierte una ruta relativa en absoluta por sí solo.

Files

Es una clase de utilidad con las acciones (borrar, copiar, mover, leer, etc) que podemos realizar sobre las rutas (`Path`).

Métodos principales de Files

Método	Descripción
<code>exists()</code> , <code>isDirectory()</code> , <code>isRegularFile()</code> , <code>isReadable()</code>	Verificar de existencia y accesibilidad.
<code>list()</code> , <code>walk()</code>	Listar contenido de un directorio.
<code>readAttributes()</code>	Obtener atributos (tamaño, fecha, etc.).
<code>createDirectory()</code>	Crear un directorio: Solo crea el directorio y espera que todo el "camino" hasta él ya exista.
<code>createDirectories</code>	Crea un directorio y también los directorios padre si no existen. Es la forma más segura.
<code>createFile()</code>	Crear un fichero.



Método	Descripción
<code>delete()</code>	Borrar un fichero o directorio (lanza una excepción si el borrado falla). Lanza la excepción <code>NoSuchFileException</code> si el fichero o directorio no existe. Es más seguro <code>deleteIfExists()</code> .
<code>move(origen, destino)</code>	Mover o renombrar un fichero o directorio.
<code>copy(origen, destino)</code>	Copiar un fichero o directorio. Si el destino ya existe se puede sobrescribir utilizando <code>copy(Path, Path, REPLACE_EXISTING)</code> . Si se copia un directorio no se copiará su contenido, el nuevo directorio estará vacío.

Ejemplo 2

Partimos de una carpeta llamada `muestras` donde guardamos fotos, descripciones de texto y registros de audio de la naturaleza sin ningún orden. Este programa organizará automáticamente los archivos en subcarpetas según su formato (extensión) para que el herbario quede perfectamente estructurado.

Puedes descargar la carpeta del ejemplo comprimida desde el siguiente enlace: [muestras.zip](#)). La carpeta debe estar ubicada en la raíz del proyecto de IntelliJ (al mismo nivel que la carpeta `src` y que el archivo `build.gradle.kts`).

```

1  import java.nio.file.Path
2  import java.nio.file.Files
3  import java.nio.file.StandardCopyOption
4  import kotlin.io.path.extension // Extensión de Kotlin para obtener la
5  extensión
6
7  fun main() {
8      organizar()
9  }
10
11 fun organizar(){
12     // 1. Ruta de la carpeta de muestras botánicas a organizar
13     val carpeta = Path.of("muestras")
14
15     println("--- Iniciando la clasificación botánica en la carpeta: " +
16 carpeta + " ---")
17     try {
18         // 2. Recorrer la carpeta desordenada y utilizar .use para
19         asegurar el cierre de recursos
20         Files.list(carpeta).use { streamDePaths ->
21             streamDePaths.forEach { pathFichero ->
22                 // 3. Solo nos interesan los ficheros de muestras,
23                 ignoramos subcarpetas

```

```
24         if (Files.isRegularFile(pathFichero)) {
25
26             // 4. Obtener la extensión del fichero (ej: "jpg",
27             "txt", "mp3")
28             val extension = pathFichero.extension.lowercase()
29             if (extension.isBlank()) {
30                 println("-> Ignorando fichero sin tipo: " +
31                 pathFichero.fileName)
32                 return@forEach // Salta a la siguiente muestra
33             }
34
35             // 5. Crear la ruta de destino dentro de la carpeta
36             correspondiente
37             val carpetaDestino = carpeta.resolve(extension)
38
39             // 6. Crear el directorio de destino de la categoría
40             si no existe
41             if (Files.notExists(carpetaDestino)) {
42                 println("-> Creando nueva sección para ficheros:
43                 ." + extension)
44                 Files.createDirectories(carpetaDestino)
45             }
46
47             // 7. Mover la muestra botánica a su nueva ubicación
48             clasificada
49             val pathDestino =
50             carpetaDestino.resolve(pathFichero.fileName)
             Files.move(pathFichero, pathDestino,
             StandardCopyOption.REPLACE_EXISTING)
             println("-> Clasificando " + pathFichero.fileName + "
             en la carpeta (." + extension + ")")
             }
         }
         println("\n--- ¡Clasificación del herbario completada con éxito!
         ---")
     } catch (e: Exception) {
         println("\n--- Ocurrió un error durante la clasificación de
         muestras ---")
         e.printStackTrace()
     }
 }
```



Prueba y analiza el ejemplo

Prueba el código de ejemplo y verifica que la salida por consola es:

```
1 --- Iniciando la clasificación botánica en la carpeta: muestras ---
2 -> Creando nueva sección para ficheros: .txt
3 -> Clasificando flor.txt en la carpeta (.txt)
4 -> Clasificando arbusto.txt en la carpeta (.txt)
5 -> Creando nueva sección para ficheros: .jpg
6 -> Clasificando 20191106_071048.jpg en la carpeta (.jpg)
7 -> Clasificando 20191101_071830.jpg en la carpeta (.jpg)
8 -> Creando nueva sección para ficheros: .mp3
9 -> Clasificando pad-harmonious-and-soothing-voice-like-background.mp3 en
10 la carpeta (.mp3)
11 -> Clasificando dark-cinematic-atmosphere.mp3 en la carpeta (.mp3)
12 -> Creando nueva sección para ficheros: .mp4
13 -> Clasificando 293968_small.mp4 en la carpeta (.mp4)
14 -> Creando nueva sección para ficheros: .pdf
15 -> Clasificando lorem-ipsuam-1.pdf en la carpeta (.pdf)
16 -> Clasificando lorem-ipsuam-2.pdf en la carpeta (.pdf)
17 --- ¡Clasificación del herbario completada con éxito! ---
```



Autoevaluación

Pregunta 3: ¿Qué ocurrirá al ejecutar este código si dentro de la carpeta `proyecto/datos` existe una subcarpeta llamada `datos_fin` pero no contiene ningún fichero regular en su raíz?

```
1  import java.nio.file.Files
2  import java.nio.file.Path
3
4  fun main() {
5      val carpeta = Path.of("proyecto/datos")
6
7      Files.list(carpeta).use { stream ->
8          stream.forEach { elemento ->
9              if (Files.isRegularFile(elemento)) {
10                 println("Fichero: ${elemento.fileName}")
11             } else if (Files.isDirectory(elemento)) {
12                 println("Directorio: ${elemento.fileName}")
13             }
14         }
15     }
16 }
```

- A) Se producirá un error de compilación porque `Files.list` solo puede listar ficheros, no carpetas.
- B) El código se ejecutará sin problemas pero no mostrará nada por consola, ya que la subcarpeta `datos_fin` está vacía de ficheros.
- C) Se imprimirá por consola: `Directorio: datos_fin`.
- D) Se producirá una excepción en tiempo de ejecución porque `Files.list` entra de forma recursiva en `datos_fin` y al estar vacía falla.

” Solución

- ✗ A) `Files.list` devuelve cualquier elemento del primer nivel del directorio (tanto ficheros como carpetas), por lo que es totalmente válido y compila sin problemas.
- ✗ B) Aunque `datos_fin` no contenga ficheros en su interior, ella misma es un directorio que está en el primer nivel de `proyecto/datos`. El stream sí la detectará al evaluar el segundo condicional.
- ✓ C) Como `datos_fin` es un directorio directo dentro de `proyecto/datos`, `Files.list` lo incluye en el stream. Al no cumplir `isRegularFile` pero sí `isDirectory`, imprimirá `Directorio: datos_fin`.
- ✗ D) `Files.list` no es recursivo (esa es la función de `Files.walk`). Solo lista el contenido directo del primer nivel y nunca inspecciona el interior de subcarpetas de forma automática.

Pregunta 4: ¿Por qué es obligatorio usar `.use { }` con el stream de `Files.list()` ?



- A) Para convertir el stream en una `List<Path>` de Kotlin.
- B) Para poder encadenar operadores funcionales como `filter` y `sorted`.
- C) Para garantizar que el descriptor del sistema de ficheros se cierra automáticamente y no queda una fuga de recursos.
- D) Porque sin `.use { }` el stream devuelve solo directorios e ignora los ficheros.

” Solución

- ✗ A) `.use { }` no convierte el stream en lista. Para eso se usaría `.toList()` como operación terminal.
- ✗ B) Los operadores `filter`, `sorted`, etc. se pueden encadenar sobre el stream independientemente de si se usa o no `.use { }`.
- ✓ C) `Files.list()` abre un descriptor del sistema de ficheros que permanece abierto mientras el stream existe. Si no se cierra, se produce un *resource leak*. `.use { }` garantiza el cierre automático al salir del bloque, incluso si ocurre una excepción.
- ✗ D) `Files.list()` devuelve todos los elementos del primer nivel (ficheros y directorios). El tipo de elemento no depende del uso de `.use { }`.

Técnicas de recorrido de directorios

Como hemos visto en el clasificador anterior, recorrer directorios para "mirar" o gestionar su contenido es clave. Aquí analizamos los diferentes métodos para hacerlo:

1. `Files.list(path)` : Lista únicamente el contenido del directorio especificado **sin acceder a las subcarpetas**. Es ideal para operaciones superficiales (como nuestra clasificación anterior, donde solo queríamos trabajar a primer nivel).

- **Ventajas:**

- Rápido y eficiente al no ser recursivo.
- Control preciso, operando solo en el primer nivel del directorio.
- Devuelve un `Stream` de Java que permite usar operadores funcionales (`filter`, `map`, etc.) de forma segura con `.use`.

- **Inconvenientes:**

- No explora subdirectorios.
- Para recorrer un árbol completo, se necesita implementar la recursividad manualmente.

2. `Files.walk(path)` : Recorre un directorio y todo su contenido **recursivamente**. Entra en cada subcarpeteta y en sus subcarpetetas de manera sucesiva. Es extremadamente útil para

 búsquedas globales en el herbario (ej. buscar fotos de flores en cualquier rincón del proyecto, borrar reportes temporales o contar ficheros de registros).

- **Ventajas:**

- Recorre árboles de directorios completos (recursivo) de forma muy sencilla.
- Extremadamente potente para búsquedas profundas o para aplicar operaciones en cascada.
- Devuelve un `Stream`, permitiendo un filtrado muy expresivo.

- **Inconvenientes:**

- Puede ser lento y consumir más memoria en estructuras gigantescas con miles de subcarpetas y ficheros.
- Es una herramienta excesiva ("overkill") si solo necesitas leer el nivel superior.

3. `Files.newDirectoryStream(path)`: Es similar a `Files.list()`, pues lista solo el contenido inmediato. La diferencia es que no devuelve un `Stream` de Java 8, sino un `DirectoryStream`, una versión más antigua optimizada para bucles tradicionales `for`.

- **Ventajas:**

- Utiliza un bucle `for-each` tradicional, que puede resultar más familiar a nivel sintáctico.

- **Inconvenientes:**

- **¡PELIGRO!** Requiere cerrar el recurso manualmente (`.close()`). Si se olvida, provoca fugas de recursos (*resource leaks*).
- Es menos expresivo que los `Streams`, ya que no se pueden encadenar operadores funcionales fácilmente.
- Es preferible utilizar `Files.list().use { ... }`.

Ejemplo 3

Después de clasificar nuestros archivos, queremos crear un listado para ver como ha quedado la estructura de nuestra carpeta `muestras`. Necesitamos listar cada una de las subcarpetas de clasificación y ver qué muestras hay dentro de cada una de ellas de forma jerárquica.

```

1  import java.nio.file.Path
2  import java.nio.file.Files
3
4  fun main() {
5      listado()
6  }
7
8  fun listado(){
9      val carpetaPrincipal = Path.of("muestras")
10
11      println("--- Estructura final del Herbario Digital con Files.walk() -
12  --")

```

```

13     try {
14         Files.walk(carpetaPrincipal).use { stream ->
15             // Ordenar el stream para una visualización ordenada por
16             categorías
17             stream.sorted().forEach { path ->
18                 // Calcular profundidad para la indentación
19                 // Restamos el número de componentes de la ruta base para
20                 que la raíz no tenga sangrado
21                 val profundidad = path.nameCount -
22                 carpetaPrincipal.nameCount
23                 val indentacion = "\t".repeat(profundidad)
24
25                 // Determinamos si es una sección (categoría/directorio)
26                 o un registro (fichero)
27                 val prefijo = if (Files.isDirectory(path)) "(CATEGORÍA)"
28                 else "(MUESTRA)"
29
30                 // No imprimimos la propia carpeta raíz, solo su
31                 contenido clasificado
32                 if (profundidad > 0) {
33                     println("$indentacion$prefijo ${path.fileName}")
34                 }
35             }
36         }
37     } catch (e: Exception) {
38         println("\n--- Ocurrió un error durante la generación del informe
39         botánico ---")
40         e.printStackTrace()
41     }
42 }

```



Prueba y analiza el ejemplo

Prueba el código de ejemplo y verifica que la salida por consola es:

```

1  --- Estructura final del Herbario Digital con Files.walk() ---
2  (CATEGORÍA) jpg
3      (MUESTRA) 20191101_071830.jpg
4      (MUESTRA) 20191106_071048.jpg
5  (CATEGORÍA) mp3
6      (MUESTRA) dark-cinematic-atmosphere.mp3
7      (MUESTRA) pad-harmonious-and-soothing-voice-like-background.mp3
8  (CATEGORÍA) mp4
9      (MUESTRA) 293968_small.mp4
10 (CATEGORÍA) pdf
11     (MUESTRA) lorem-ipsum-1.pdf
12     (MUESTRA) lorem-ipsum-2.pdf
13 (CATEGORÍA) txt
14     (MUESTRA) arbusto.txt
15     (MUESTRA) flor.txt

```



Autoevaluación

Pregunta 5: ¿Por qué en el código del Ejemplo 3 se realiza la resta `path.nameCount - carpetaPrincipal.nameCount` para calcular la indentación?

- A) Para evitar que el directorio raíz `multimedia` aparezca con sangrado y que los elementos directamente dentro de él tengan una sola tabulación.
- B) Para que el programa conozca el tamaño en bytes del fichero antes de calcular el espaciado.
- C) Para detectar si el elemento es un fichero o un directorio antes de asignar el prefijo.
- D) Para ordenar los elementos del stream de mayor a menor profundidad antes de imprimirlos.

” Solución

- ✓ A) La resta `path.nameCount - carpetaPrincipal.nameCount` calcula cuántos niveles de profundidad hay entre el elemento actual y el directorio raíz. Si el resultado es 0, el elemento ES la carpeta raíz (no se imprime). Si es 1, está a un nivel de profundidad y se indenta con un tabulador; si es 2, con dos, y así sucesivamente.
- ✗ B) `nameCount` devuelve el número de componentes de la ruta (fragmentos de texto separados por `/`), no el tamaño en bytes del fichero. Para eso existe `Files.size(path)`.
- ✗ C) La detección de si es fichero o directorio se realiza con `Files.isDirectory(path)`, no con la resta de `nameCount`.
- ✗ D) El stream ya está ordenado con `.sorted()` antes de aplicar esta lógica. La resta sirve exclusivamente para calcular la indentación.

Pregunta 6: ¿Cuál es la principal diferencia entre `Files.list()` y `Files.walk()`?

- A) `Files.list()` devuelve un `Stream`, mientras que `Files.walk()` devuelve una `List`.
- B) `Files.walk()` recorre recursivamente todos los subdirectorios; `Files.list()` solo lista el nivel inmediato.
- C) `Files.list()` incluye el propio directorio raíz en el stream, mientras que `Files.walk()` no.
- D) `Files.walk()` solo funciona con directorios vacíos.

**” Solución**

- ☒ A) Ambos métodos devuelven un `Stream<Path>`. Ninguno devuelve una `List` directamente.
- ☒ B) La diferencia clave es la recursividad: `Files.walk()` desciende a todas las subcarpetas (y a las subcarpetas de las subcarpetas...); `Files.list()` se detiene en el primer nivel del directorio indicado.
- ☒ C) Ambos métodos incluyen el directorio raíz como primer elemento del stream. En el Ejemplo 3 se excluye manualmente comprobando que la profundidad sea mayor que 0.
- ☒ D) `Files.walk()` funciona con cualquier directorio, esté o no vacío.

 Práctica 2: Directorios y comprobaciones



Práctica 2: Directorios y comprobaciones

Prepara **la estructura de tu proyecto**. Crea la ruta `proyecto/datos`. Basándote en los ejemplos anteriores, **desarrolla un programa** que haga lo siguiente:

1. **Defina dos rutas:** una para una carpeta llamada `datos_ini` y otra para una carpeta llamada `datos_fin` (ambas dentro de la carpeta `proyecto/datos` de tu proyecto).
2. **Comprueba los directorios:** Si las carpetas no existen, las deberá crear utilizando `Files.createDirectories`.
3. **Añade el fichero de datos:** Crea manualmente (y vacío por ahora) el fichero `videojuegos.csv` dentro de la carpeta `datos_ini`. Este fichero lo rellenarás en la Práctica 3.
4. **Comprueba ficheros:** Comprueba si el fichero `videojuegos.csv` existe dentro de `datos_ini` e imprime por consola la estructura de directorios y ficheros.

La salida de tu programa debe ser parecida a esta, la primera vez que se ejecuta:

```
1  CREACIÓN DE RUTAS PROYECTO
2  Creando rutas...
3  Creación de ruta para DATOS_INI
4  Creación de ruta para DATOS_FIN
5  MOSTRANDO ESTRUCTURA DE DIRECTORIOS Y FICHEROS
6  [DIR] datos
7  [DIR] datos_fin
8  [DIR] datos_ini
```

Y la segunda vez que se ejecuta, tras crear el fichero `videojuegos.csv`:

```
1  CREACIÓN DE RUTAS PROYECTO
2  Creando rutas...
3  MOSTRANDO ESTRUCTURA DE DIRECTORIOS Y FICHEROS
4  [DIR] datos
5  [DIR] datos_fin
6  [DIR] datos_ini
7  [FILE] videojuegos.csv
```